Практическое задание 8. Матрицы и динамическая память

Работа про динамическую память с матрицами. У каждой темы есть лестница из трех уровней: базовая реализация, динамическая память и надстройка сверху. Можно остановиться на любом шаге - оценка соответствует тому уровню, который вы закрыли.

Вайбкод (опционально): прежде чем писать код, попросите ИИ выступить архитектором: пусть он опишет, из каких функций и шагов будет состоять программа и какие данные, где хранятся, но без кода и мелких подробностей, можно блок схемами или текстом. Любая модель легко напишет готовое решение целиком, но смысл задания не в этом: возьмите предложенную архитектуру и попробуйте реализовать ее самостоятельно.

Как рисовать кадры. Если выводится не один результат, а анимация (песок, пожар, змейка), кадры не должны идти друг под другом - экран будет уезжать вниз. Перед печатью нового кадра стирайте старый:

```
printf("\033[H\033[J");   // курсор в левый верхний угол и очистка экрана
```

Как это читать: \033[- начало управляющей команды, H - поставить курсор в позицию 1,1, J - стереть все ниже курсора. Эквивалент: printf("\033[2J\033[H");. В Linux работает в любом терминале, в Windows Terminal тоже, а в старом cmd.exe придется вызывать system("cls").

И две мелочи: после вывода кадра делать fflush(stdout);, иначе буфер отдаст его рывками, и ставить паузу между кадрами – nanosleep/usleep на 0.05 с, иначе кадры пролетят мгновенно.

Как замерять время. Для честного замера нужны clock_gettime и тип struct timespec из <time.h> (это готовая структура из библиотеки, свои объявлять не нужно):

```
static double now_s(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}
```

Пользоваться так: double t0 = now_s(); multiply(a, b, c, n); double dt = now_s() - t0;

- мерить только сам алгоритм: ввод, заполнение и печать - вне таймера;
- каждый размер прогнать 3-5 раз и взять минимум, иначе поймаешь шум от системы;
- оба варианта компилировать с одинаковыми флагами, иначе сравнение нечестное;
- результат должен использоваться (например, напечатать контрольную сумму), иначе оптимизатор может выкинуть цикл.

Почему не time(NULL) и не clock(): time дает целые секунды - слишком грубо, а clock() считает процессорное время процесса, а не реальное.

Выбор варианта

Темы неравноценные: попроще заканчиваются на четверке, сложные могут доходить до пятерки.

№	Тема	Оценки	Сложность
1	Логическая игра 2048	3-4-5	4
2	Головоломка Пятнашки	3-4	3
3	Поворот матрицы на 90 градусов	3-4	2
4	Свёртка и ASCII-картинка	3-4-5	3
5	Физическая симуляция песка	3-4-5	3
6	Логическая игра Сапёр	3-4-5	3-4
7	Моделирование лесного пожара	3-4-5	3
8	Аркада Змейка	4-5	4
9	Моделирование MIMO-антенн 5G	4-5	4
10	Трассировка 5G-лучей в городе	4-5	5

Что нужно знать

- ***malloc, calloc, realloc, free***: как выделять и как освобождать память. Указатели.
- Индексация матрицы и адресная арифметика.
- Разбиение программы на функции: выделение, заполнение, печать, обработка, освобождение.

Темы

1. Логическая игра 2048

Что получается. Консольная игра: поле 4x4, после каждого хода печатается сетка и счет. Ход сдвигает все плитки в выбранную сторону, одинаковые сливаются, в освободившуюся клетку появляется 2 или 4. Игра заканчивается, когда ходов не осталось.

```
   2   4   8  16
   0   0   0   0
   0   0   0   0
   0   0   0   0
score: 30
```

Обозначения: 0 - пустая клетка, числа - плитки, score - счет.

На 3. Поле int a[4][4]. Одна функция сдвига строки влево: ненулевые элементы сдвигаются к краю, одинаковые соседние сливаются, после слияния снова сдвиг. Новая плитка ставится в случайную пустую клетку. Счет растет на сумму слитых плиток.

На 4. Поле NxN, размер из argv, есть динамическая память. Все четыре направления работают через ту же одну функцию сдвига: строку разворачивают, сдвигают и разворачивают обратно (для столбцов - транспонируют). Проверка конца игры: нет пустых клеток и нет соседних одинаковых.

На 5. Плюс история ходов с откатом: перед ходом матрица копируется в динамический массив int *history, который растет через realloc, и по команде можно вернуться на шаг назад.

2. Головоломка Пятнашки

Что получается. Игра: поле с числами от 1 до 15 и пустой клеткой. Ход меняет пустую клетку с соседней, цель - расставить числа по порядку.

```
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14  0 15
moves: 42
```

Обозначения: числа - плитки, 0 - пустая клетка, moves - число ходов.

На 3. Поле 4x4, числа и ноль. Перемешивание не случайной перестановкой, а серией допустимых ходов, иначе половина раскладов нерешаема. Ход - обмен нуля с соседом по WASD, проверка победы, счетчик ходов.

На 4. Размер NxN из ввода, динамическая память. Проверка решаемости по четности числа перестановок: на нерешаемой доске программа отказывается стартовать и предлагает перемешать заново.

3. Поворот матрицы на 90 градусов

Что получается. Одна и та же матрица до и после поворота: программа печатает обе и проверяет, что четыре поворота возвращают исходную.

```
было:       стало:
1 2 3       7 4 1
4 5 6       8 5 2
7 8 9       9 6 3
```

На 3. Матрица 4x4, заполнение с клавиатуры или случайными числами. Поворот в новую матрицу по формуле b[j][n - 1 - i] = a[i][j]. Вывод обеих матриц и проверка на четырех поворотах подряд.

На 4. n из ввода, обе матрицы через динамическую память. Меню: поворот на 90, 180 и 270. Дополнительно получить поворот другим путем (транспонирование плюс отражение) и убедиться, что результат совпадает.

4. Свёртка и ASCII-картинка

Что получается. Матрица яркостей печатается символами (чем ярче, тем плотнее символ), потом к ней применяется усреднение 3x3, и картинка печатается снова - видно, как она размывается.

```
до:             после усреднения:
0   0   0   0    0   0   0   0
0 255 255   0    0 128 128   0
0 255 255   0    0 128 128   0
0   0   0   0    0   0   0   0
```

Числа - яркость клетки от 0 до 255, показано до и после усреднения.

На 3. Матрица 8x8 со значениями 0..255 (градиент, шахматка или rand()), свёртка 3x3 усреднением, вывод символами .:-=+*#%@ по яркости.

На 4. Размер из argv, динамическая память. Аккуратные края: у крайних клеток окно обрезается и усредняется меньше соседей, за пределы матрицы программа не читает. Вывод "до и после".

На 5. Плюс разделимая свёртка в два прохода: сначала по строкам, потом по столбцам. Замер времени наивной и разделимой версии, таблица сравнения на размерах 128, 256, 512.

5. Физическая симуляция песка

Что получается. Кадры поля: песок сыплется сверху и постепенно собирается в кучу, обтекая препятствия. Каждый тик печатается новый кадр.

```
................
.......#........
......###.......
.....#####......
....##.######...
```

Обозначения: . - пусто, # - песок.

На 3. Поле 20x20, ноль - пусто, единица - песок. Каждый тик сверху в случайный столбец добавляется частица, песок падает вниз, если под ним пусто. N тиков, вывод символами.

На 4. Поле NxM, размеры из argv, препятствия (число 2), динамическая память. Матрица обходится снизу вверх (иначе за один тик частица пролетает пол-поля), при занятой клетке снизу проверяются диагонали влево и вправо.

На 5. Плюс второй материал - вода (число 3): она тоже падает, но растекается в стороны сильнее. Замер времени на большом поле (500x500) и вывод с ANSI-цветом.

6. Логическая игра Сапёр

Что получается. Поле, где открытые клетки показывают, сколько мин вокруг, закрытые помечены точкой. Попал на мину - проиграл, открыл все безопасные - победил.

```
  1 1 1 . .
  1 2 1 . .
  1 1 1 . .
  . . . . .
  . . . . .
mines: 3, flags: 1
```

Обозначения: цифра - сколько мин вокруг, . - закрытая клетка, mines - всего мин, flags - флажков.

На 3. Поле 9x9 фиксированного размера с минами. При открытии клетки вложенными циклами по соседям считается, сколько мин вокруг (с проверкой границ, чтобы не выйти за матрицу). Вывод поля с цифрами, попадание на мину - поражение.

На 4. Размер поля и число мин из scanf. Две динамические матрицы: скрытая (где мины) и видимая (что открыто). Флажки по команде, победа, когда открыты все клетки без мин.

На 5. Плюс **рекурсивное** открытие пустых областей: если вокруг клетки ноль мин, соседи открываются **рекурсивно**. Первый ход гарантированно без мины: мины расставляются после первого открытия. В конце - статистика: открыто клеток, поставлено флажков, осталось мин.

7. Моделирование лесного пожара

Что получается. Кадры леса: деревья, огонь, выгоревшая пустота и вода. Видно, как огонь расползается и через сколько тиков лес выгорает.

```
T T T . T T . T
T t t T T . T T
. t . . T T T .
~ ~ . T T . . T
```

Обозначения: T - дерево, t - огонь, . - пусто, ~ - вода.

На 3. Поле 20x20, два состояния: дерево и огонь. Каждый тик огонь поджигает соседние деревья, сам через тик гаснет. N тиков, вывод каждого кадра.

На 4. Четыре состояния (пусто, дерево, огонь, вода), поджог с вероятностью через rand(). Два кадра - текущий и следующий, иначе обновление портит соседей. Динамическая память, статистика сгоревших деревьев.

На 5. Плюс ветер: огонь распространяется в основном в одну сторону (например, поджигает только правого и нижнего соседа). Параметры из argv, статистика по тикам (сколько горит и сколько сгорело) и замер времени.

8. Аркада Змейка

Что получается. Кадры игрового поля: змейка ползет, ест еду и растет, счет увеличивается. Столкновение со стеной или с собой заканчивает игру.

```
+----------------+
|                |
|    ooo#        |
|                |
|            *   |
+----------------+
```

Обозначения: o - тело, # - голова, * - еда.

На 4. Поле NxM с динамической памятью. Тело хранится как массив номеров клеток (y * cols + x) фиксированной емкости: голова добавляет новый номер, хвост стирается. Движение по команде, столкновения со стеной и с собой, счет.

На 5. Плюс рост тела через realloc (массив номеров клеток увеличивается при поедании), ускорение по мере роста и вывод счета после каждого кадра.

9. Моделирование MIMO-антенн 5G

Что получается. Карта мощности сигнала символами разной плотности и строка с координатами максимума: видно, куда "сфокусировался" луч.

```
. . : : + + # # @ @
. : : + + # # @ @ @
. : + + # # @ @ @ @
. : : + + # # @ @ @
. . : : + + # # @ @
max: row 2, col 6
```

Плотность символа показывает мощность сигнала: . - минимум, @ - максимум, max - координаты лучшего пикселя.

Формулы. Антенная решетка - это M точек с координатами ax[m], ay[m]. Для каждой клетки карты (i, j) считаем:

- расстояние до антенны: d = sqrt((xi - ax[m])^2 + (yj - ay[m])^2);
- фазу: phi = 2 * pi * d / lambda; для игрушечной модели можно взять lambda = 1;
- сумму: Re = Re + cos(phi), Im = Im + sin(phi), мощность S = Re * Re + Im * Im.

Это то же самое, что |sum exp(i * phi)|^2, только без комплексных чисел: косинусы и синусы складываются отдельно.

Как это ложится на кадры. Кадр - один проход по всей матрице: два вложенных цикла по i и j, внутри цикл по антеннам m. Для каждой клетки получается одно число S, его нормируем на максимум и печатаем символом. Для карты 40x40 и 8 антенн это 12800 сложений на кадр, поэтому кадры можно печатать подряд, меняя число антенн или их положение.

Псевдокод кадра.

```
for (frame = 0; frame < frames; frame++) {
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) {
            Re = 0; Im = 0;
            for (k = 0; k < antennas; k++) {
                // Формулы
            }
            // Расчет S[i][j]
        }
    print_map(S);                  // нормировка на максимум и символы
    printf("\033[H\033[J");       // стереть кадр перед следующим
    delay_ms(100);
}
```

На 4. Сетка фиксированного размера, антенная решетка задается двумя массивами координат ax[] и ay[]. Для каждой клетки считается сумма сигналов от всех антенн с учетом фазы (формулу можно упростить), результат пишется в double **, выводится символами, отдельно ищется максимум.

На 5. Плюс сравнение карт для разного числа антенн (2, 4 и 8): видно, как сужается луч. Замер времени расчета и вывод обеих карт рядом.

10. Трассировка 5G-лучей в городе

Что получается. Карта города, где стены - решетка, а пройденный лучом путь отмечен звездочками: упрощенная карта покрытия радиоволн.

```
###########
#.........#
#..*.....*#
#.*.......#
#.........#
###########
```

Обозначения: # - стена, * - путь луча (покрытие).

Идея в двух словах. Луч - это точка, которая медленно едет по карте. На каждом шаге смотрим, в какую клетку он попал: воздух - идем дальше, стекло или бетон - теряем часть мощности и отражаемся. Где луч прошел, там накапливаем мощность - так и получается карта покрытия.

Формулы. Направление задается углом и переводится в вектор, дальше работают только координаты:

- направление: dx = cos(alpha), dy = sin(alpha); это единичный вектор, его длина равна 1;

- шаг: x = x + dx * h, y = y + dy * h; h - размер шага, 0.5 клетки дают достаточную точность;
- клетка под лучом: cx = (int)x, cy = (int)y - дробную часть просто отбрасываем;
- стена: если изменился только cx, удар в вертикальную стену, dx = -dx; если только cy, в горизонтальную, dy = -dy; если обе, попали в угол - меняются обе;
- мощность: power = power * k, где k - коэффициент материала клетки;
- карта: cover[cy][cx] = cover[cy][cx] + power - именно сложение, а не присваивание.

Почему отражение - это смена знака. Стенки у нас выровнены по сетке, поэтому общий закон отражения V' = V - 2 * (V * n) * n превращается в смену знака одной компоненты: для вертикальной стены нормаль (1, 0), для горизонтальной (0, 1). Никакой тригонометрии в момент удара не нужно, угол падения равен углу отражения автоматически.

Материалы. Коэффициент k зависит от того, что за материал в клетке. За один шаг мощность умножается на него.

Материал	**k за шаг**
воздух	0.99
стекло	0.8
бетон	0.5

Как это ложится на кадры. Один кадр - это либо один луч, прошедший K шагов, либо вся карта после одного шага: реши, что интереснее показывать. Условие остановки простое: вышли за границы, мощность упала ниже одного процента или кончились K шагов. Перед печатью карту нормируем на максимум и переводим в символы - так же, как в теме со свёрткой.

Частые ошибки.

- луч застревает в стене: после отражения сдвинь точку назад на h;
- отражение не по той оси: проверяй, какая именно координата изменилась;
- мощность не падает: забыл умножить на k при входе в материал;
- карта пустая: в клетку записывают power вместо прибавления.

На 4. Карта NxN с динамической памятью: 0 - воздух, 1 - бетон, 2 - стекло. Луч стартует из заданной точки с направлением dx и dy, шагает по клеткам, при столкновении со стеной отражается (меняется dx или dy), после K шагов рисуется карта покрытия.

На 5. Плюс несколько лучей под разными углами, разное ослабление у материалов (бетон сильнее, стекло слабее), замер времени и итоговая карта покрытия от всех лучей.

Под Звездочкой (+ балл):

Что получается. Таблица замеров и график в консоли, по которому видно, что умножение с транспонированной B быстрее в несколько раз. Объяснение, почему так, тоже часть работы.

Что сделать.

- Реализовать два умножения квадратных матриц: наивное ijk (строка A на столбец B) и ikj с транспонированной B.

- Замерить время на 4 размерах: 128, 256, 512, 1024. Время брать через clock_gettime(CLOCK_MONOTONIC), каждый размер прогнать хотяб 2-3 раза и взять лучший результат.
- Собрать таблицу: размер, число элементов n * n, время обоих вариантов, ускорение.
- Построить график: по оси Y время, полоса из символов длиной пропорционально времени, отдельные ряды для наивного и оптимизированного умножения. График не обязательно рисовать в консоли: подойдут Excel, Python, любой вайбкод - важно, чтобы по нему было видно, как время растет с числом элементов.

Пример таблицы.

N	**Наивное, с**	**С транспонированием, с**	**Ускорение**
128	0.004	0.002	2.0x
256	0.031	0.011	2.8x
512	0.245	0.078	3.1x
1024	1.960	0.590	3.3x

Что объяснить. Почему ijk медленнее: B читается по столбцам, то есть программа прыгает по памяти, а ikj идет подряд. Дело не в количестве операций, а в кэше и в том, что матрица лежит по строкам.